# Raft, one failure at a time

Speaker notes · D7024E seminar

Review of Ongaro & Ousterhout, In Search of an Understandable Consensus Algorithm, USENIX ATC 2014 · raft.ronstad.se

**How to use:** press → for each click. Each click below says what appears on screen and what to say. In the browser, press **S** for the speaker view (notes, timer, and a light that shows when the animation is done). Wait for “done” before clicking on demo slides.

### Timing plan

| # | Slide | Min | Total |
|---|---|---|---|
| 1 | Raft, one failure at a time | 0:30 | 0:30 |
| 2 | Why this paper? | 1:30 | 2:00 |
| 3 | How they designed for understandability | 1:00 | 3:00 |
| 4 | A replicated, deterministic log | 1:00 | 4:00 |
| 5 | Everyone accepts writes | 1:00 | 5:00 |
| 6 | Why a majority? **(optional)** | 1:00 | 6:00 |
| 7 | Electing a leader, naively | 1:00 | 7:00 |
| 8 | Randomized election timeouts | 1:30 | 8:30 |
| 9 | Replicating the log, fire-and-forget | 1:00 | 9:30 |
| 10 | Say where it goes, acknowledge, retry | 1:30 | 11:00 |
| 11 | Applying too early | 1:30 | 12:30 |
| 12 | Commit before you apply | 1:30 | 14:00 |
| 13 | Catching up a returning server | 1:00 | 15:00 |
| 14 | The wrong server wins | 1:30 | 16:30 |
| 15 | Vote only for up-to-date logs | 1:30 | 18:00 |
| 16 | Two leaders? Network partitions | 1:30 | 19:30 |
| 17 | The subtle one (Figure 8) **(optional)** | 1:30 | 21:00 |
| 18 | Only count replicas for the current term **(optional)** | 1:00 | 22:00 |
| 19 | How the authors evaluated Raft | 1:00 | 23:00 |
| 20 | Students understood Raft better | 1:30 | 24:30 |
| 21 | Correctness: specified, (mostly) proven | 1:00 | 25:30 |
| 22 | Leader election is fast, with randomness | 1:00 | 26:30 |
| 23 | What the authors conclude | 1:00 | 27:30 |
| 24 | What we think | 2:00 | 29:30 |
| 25 | That’s Raft | 1:00 | 30:30 |
| 26 | Sandbox | 2:00 | 32:30 |

Optional slides can be skipped to save about 3:30 min.

## 1. Raft, one failure at a time

· about 0:30 min

> **WHEN THE SLIDE OPENS**
>
> Hi. We are presenting the Raft paper by Ongaro and Ousterhout. Plan: why the paper exists, then we rebuild Raft live, one failure at a time, then how the authors tested it, their results, and what we think.

**Background:** Plan: why the paper exists, then we rebuild Raft live one failure at a time, then how the authors evaluated it, results, and our critique.

## 2. Why this paper?

Introduction · Introduction · about 1:30 min

> **WHEN THE SLIDE OPENS**
>
> Start with the problem the paper attacks.

> **CLICK 1 · on screen:** Replicated systems keep copies of data on many machines, which must agree on every change. That is consensus.
>
> Any database or config store with copies on many machines needs this. Kubernetes stores its state this way.

> **CLICK 2 · on screen:** For decades the answer was Paxos: proven correct, but notoriously hard to understand and hard to build on.
>
> Paxos is the classic solution. Even the authors needed almost a year to understand all of it. Point at the Google quote.

> **CLICK 3 · on screen:** Research question: can we design a consensus algorithm whose primary goal is understandability?
>
> This is the research question. Understandability is unusual as a primary goal in systems research.

> **CLICK 4 · on screen:** Their answer is Raft. Students learned it better than Paxos, it is formally specified, and it performs as well.
>
> Say the key results now: students did better on Raft, it is formally specified, and it is as fast. We show the details later.

**Background:** Key results up front (the tips ask for this). The quote is from Google’s Chubby team, cited in the paper as [4].

## 3. How they designed for understandability

Methods · design · Methods · about 1:00 min

> **CLICK 1 · on screen:** Decomposition: split consensus into three mostly independent problems.
> Three parts you can learn one at a time: leader election, log replication, safety.

> **CLICK 2 · on screen:** Shrink the state space: fewer ways for servers to disagree, fewer cases to reason about.
> Examples: one strong leader instead of equal peers, logs that can’t have holes, and randomness instead of complicated tie-break rules.

> **CLICK 3 · on screen:** Next we rebuild Raft from nothing, one failure at a time. Every rule we add is from the paper.
> From here on the simulator does the talking. Every rule we add is from the paper; the sidebar on the right collects them.

**Background:** This is the artifact part of the paper (Sections 3–5). The live demo is our way of presenting it.

## 4. A replicated, deterministic log

Methods · the artifact, rebuilt live · The goal · about 1:00 min

> **WHEN THE SLIDE OPENS**
> Five servers, each with a log and a tiny key-value store.

> **CLICK 1 · on screen:** Five servers run the same deterministic state machine: a tiny key–value store.
> Deterministic means: same input, same output. No randomness, no clocks inside the state machine.

> **CLICK 2 · on screen:** Every command goes into a log; each server applies its log in order.
> Watch the client send three writes. Each server appends them and applies them in order.

> **CLICK 3 · on screen:** Same commands in the same order ⇒ same state on every server.
> So the whole problem becomes: agree on the log.

> **CLICK 4 · on screen:** Goal: keep working while any minority of servers is down or unreachable.
> And we want to survive failures. With five servers, any two can be down.

**Point at:** The logs table and the state machines filling up identically.

**Background:** Deterministic is the key word: if the input sequence is identical, the output is identical. So the whole problem reduces to agreeing on the log.

## 5. Everyone accepts writes

Methods · the artifact, rebuilt live · Problem · about 1:00 min

> **WHEN THE SLIDE OPENS**
>
> First idea: let every server accept writes.

> **CLICK 1 · on screen:** Two clients write the same key at the same moment.
>
> Client A on the left writes x=1, client B on the right writes x=2, at the same time.

> **CLICK 2 · on screen:** Messages travel different distances, so servers see them in different orders…
>
> Messages travel different distances, so the left servers get A first and the right servers get B first.

> **CLICK 3 · on screen:** …and end up with different values for x. The replicas have diverged.
>
> Point at the red values: half the servers think x=1, the others x=2. Nobody did anything wrong.

> **CLICK 4 · on screen:** Fix: let one server, the leader, decide the order of everything.
>
> So someone has to decide the order. That’s the leader.

**Point at:** Left vs right servers; red cells in the state machines.

**Background:** Point at the left servers vs the right servers: left got A first, right got B first. Nobody did anything wrong. We need someone to decide the order.

## 6. Why a majority? (optional)

Methods · the artifact, rebuilt live · Background · about 1:00 min

> **WHEN THE SLIDE OPENS**
>
> Optional slide. Skip it if short on time.

> **CLICK 1 · on screen:** Decisions need a majority: 3 of 5 servers.
>
> Every decision in Raft needs three of five.

> **CLICK 2 · on screen:** Any two majorities overlap in at least one server…
>
> Blue is one majority, orange is another. They always share at least one server, here S3.

> **CLICK 3 · on screen:** …and that server remembers the first decision, so a conflicting second one can’t also win.
>
> That shared server is why two conflicting decisions can’t both win. Every safety rule later uses this.

> **CLICK 4 · on screen:** 5 servers survive 2 failures: 3 left is still a majority.
>
> Two servers crash. Three are left, still a majority, so the cluster keeps working.

**Background:** Blue majority S1–S3, orange majority S3–S5: they always share someone. This overlap argument is behind every safety rule later.

## 7. Electing a leader, naively

Methods · the artifact, rebuilt live · Problem · about 1:00 min

> **WHEN THE SLIDE OPENS**
>
> We need a leader. How do we pick one?

> **CLICK 1 · on screen:** No leader yet. A server that hears nothing for a while becomes a candidate and asks for votes.
>
> If a server hears nothing for a while, it assumes there is no leader and runs for election.

> **CLICK 2 · on screen:** Each election has a number, the term. A server votes at most once per term; a majority of votes wins.
>
> Terms are like election numbers. One vote per server per term, and a majority wins.

> **CLICK 3 · on screen:** Everyone uses the same timeout…
>
> Now give every server the same timeout. Watch the timer rings.

> **CLICK 4 · on screen:** …so everyone becomes a candidate at once and votes for itself. Split vote. Again. Forever.
>
> Everyone times out together, everyone votes for itself, nobody gets three votes. The term number keeps climbing. This never ends.

**Point at:** Term numbers climbing on all five servers; the events list.

**Background:** Watch the term number climb on every server while nobody wins. Each candidate already voted for itself, so it rejects everyone else.

## 8. Randomized election timeouts

Methods · the artifact, rebuilt live · Fix · about 1:30 min

**WHEN THE SLIDE OPENS**

The fix is almost embarrassingly simple.

**CLICK 1 · on screen:** Each server picks a random timeout (here 2.5–7.5 s).

Each server picks a random timeout. Point at the rings: they now run out at different times.

**CLICK 2 · on screen:** The first to wake up usually collects a majority before anyone else times out.

The first one to wake up asks for votes, and the others vote for it before their own timers run out.

**CLICK 3 · on screen:** The leader sends heartbeats, so followers keep resetting their timers.

The leader keeps sending small heartbeat messages. Each one refills the followers' timers, so no new election starts.

**CLICK 4 · on screen:** Kill the leader: followers time out and elect a new one in a higher term.

Now kill the leader. Heartbeats stop, a timer runs out, and we get a new leader in a higher term.

**Point at:** The crown; rings refilling with each heartbeat; the term going up after the crash.

**Background:** Ring around each follower = its timer. Heartbeats refill it. You can click servers yourself here.

## 9. Replicating the log, fire-and-forget

Methods · the artifact, rebuilt live · Problem · about 1:00 min

**WHEN THE SLIDE OPENS**

We have a leader. Now it has to copy its log to the followers.

**CLICK 1 · on screen:** S1 is leader. It appends each client command and sends it to every follower.

Simplest version: send each new command once and hope.

**CLICK 2 · on screen:** The network is unreliable: some messages are lost.

Point at the red crosses: those messages were lost.

**CLICK 3 · on screen:** Followers append whatever arrives, so one lost message shifts everything after it.

Followers just append what arrives, so after one lost message everything later sits at the wrong index.

> **CLICK 4 · on screen:** Logs (red outlines) and state machines no longer agree.
> Red outlines in the log table, red values in the state machines. The copies have diverged.

**Point at:** Red crosses on the links; S3 and S5 rows.

**Background:** S3 missed the 2nd write, S5 the 3rd. Their later entries are at the wrong index.

## 10. Say where it goes, acknowledge, retry

Methods · the artifact, rebuilt live · Fix · about 1:30 min

> **WHEN THE SLIDE OPENS**
> Same lost messages as before, three small changes.

> **CLICK 1 · on screen:** Each AppendEntries names the previous entry (index + term); a follower only appends if it has that entry.
> Every message says what entry comes right before the new ones. If the follower doesn’t have that entry, it refuses.

> **CLICK 2 · on screen:** Followers reply. The leader tracks each follower’s nextIndex (yellow underline).
> Followers answer. The yellow underline is where the leader thinks each follower is.

> **CLICK 3 · on screen:** No reply? The leader resends from nextIndex with its next heartbeat.
> No answer means the leader sends it again with the next heartbeat.

> **CLICK 4 · on screen:** Lost messages now cause delay, never divergence.
> Same losses as last slide, but now every log ends up identical. Lost messages only cost time.

**Point at:** Logs converging; all state machines equal at the end.

**Background:** Same message losses as the previous slide. The heartbeat doubles as the retry timer.

## 11. Applying too early

Methods · the artifact, rebuilt live · Problem · about 1:30 min

> **WHEN THE SLIDE OPENS**
> So far a server applies an entry as soon as it has it. That is a problem.

> **CLICK 1 · on screen:** Right now a server applies an entry the moment it’s in its log, and the leader answers the client immediately.
>
> Look at the bottom panel: that’s what the client was promised.

> **CLICK 2 · on screen:** A client writes. S1 applies it and says OK... its messages are lost... and it crashes.
>
> Client writes, S1 applies it and says OK, its messages get lost, and S1 crashes.

> **CLICK 3 · on screen:** A new leader is elected. It never saw that entry.
>
> Wait for the new leader. It never saw that write.

> **CLICK 4 · on screen:** The next write lands at the same index. The client was told OK, and the write is gone.
>
> The next write takes the same spot in the log. Point at the receipt: struck out, LOST. The client was told OK and the data is gone.

> **CLICK 5 · on screen:** When S1 returns, its entry is overwritten, but it already applied it.
>
> Bring S1 back: its log gets fixed, but its state machine already applied the lost value.

**Point at:** The “Client was told OK” panel.

**Background:** The receipts panel shows what the client believes happened. The struck-out one is a broken promise.

## 12. Commit before you apply

Methods · the artifact, rebuilt live · Fix · about 1:30 min

> **WHEN THE SLIDE OPENS**
>
> The fix: a write only counts once a majority has it.

> **CLICK 1 · on screen:** An entry is committed once a majority stores it. Only committed entries are applied, and only then does the client get OK.
>
> Committed means stored on a majority. Only then apply it, and only then tell the client OK.

> **CLICK 2 · on screen:** Same story: S1 takes a write, its messages are lost, it crashes.
>
> Exact same scenario as before.

**CLICK 3 · on screen:** It never committed, so the client never got OK and will retry. No promise was broken.

S1 never got a majority, so the client never got OK. The client knows to retry. No broken promise.

**CLICK 4 · on screen:** Followers learn the commit index from the leader's next AppendEntries (faded = not yet committed).

Faded entries are not yet committed. Followers learn the commit point from the leader's next message.

**Point at:** Faded vs solid cells; the receipts panel stays clean.

**Background:** Uncommitted entries are drawn faded and dashed. Majority storage means any future leader will have it (we'll need one more rule for that).

## 13. Catching up a returning server

Methods · the artifact, rebuilt live · Demo · about 1:00 min

**WHEN THE SLIDE OPENS**

What happens when a server comes back after being away?

**CLICK 1 · on screen:** S4 was offline while entries 3–6 were committed, and S1 has just become leader.

S4 missed four entries, and S1 became leader while it was gone.

**CLICK 2 · on screen:** A new leader doesn't know how far behind anyone is: it guesses nextIndex = end of its own log.

A new leader just guesses: everyone is up to date.

**CLICK 3 · on screen:** S4 comes back. AppendEntries is rejected: S4 doesn't have the previous entry.

S4 comes back and says no: it doesn't have the entry before the new ones.

**CLICK 4 · on screen:** The leader steps nextIndex back one at a time until the logs match…

Point at the yellow underline in S4's row: it steps left, one entry per rejection.

**CLICK 5 · on screen:** …then streams the missing entries, and S4 rebuilds its state machine from the log.

Once they match, the leader sends everything S4 is missing, and S4 rebuilds its key-value store.

**Point at:** S4's row and its yellow underline walking left.

**Background:** Watch the yellow underline in S4′s row walk left. Real implementations skip back faster using a hint from the follower.

## 14. The wrong server wins

Methods · the artifact, rebuilt live · Problem · about 1:30 min

**WHEN THE SLIDE OPENS**

Now a nasty one. Look at the logs: S4 and S5 are missing three committed entries.

**CLICK 1 · on screen:** S4 and S5 were down while S1–S3 committed entries 4–6 (term 2).

They were down while S1 to S3 committed entries 4 to 6.

**CLICK 2 · on screen:** S1 crashes just as S4 and S5 return. S5 times out first…

S1 crashes, S4 and S5 come back, and S5 times out first.

**CLICK 3 · on screen:** …nobody looks at S5′s log, so it wins the election.

Nobody checks how complete S5′s log is, so it gets votes and wins.

**CLICK 4 · on screen:** Its next entry overwrites committed entries on S2 and S3.

Its next entry overwrites committed entries on S2 and S3. Red banner, and the receipts show the client’s data is lost. Committed data must never disappear.

**Point at:** The red banner; LOST receipts.

**Background:** Committed meant ‘on a majority’, but the new leader wasn’t part of that majority. Committed data must never be lost.

## 15. Vote only for up-to-date logs

Methods · the artifact, rebuilt live · Fix · about 1:30 min

**WHEN THE SLIDE OPENS**

Fix: be picky about who you vote for.

**CLICK 1 · on screen:** Refuse to vote for a candidate whose log is behind yours: compare the last entry’s term, then log length.

Compare the last entry: higher term wins; same term, longer log wins. Never vote for someone behind you.

**CLICK 2 · on screen:** Same story: S1 crashes, S4 and S5 return, S5 times out first…

Same story as before.

> **CLICK 3 · on screen:** S2 and S3 refuse. S5 can’t reach a majority.
> S2 and S3 refuse. Point at the events list: “refuses vote, log is behind”.

> **CLICK 4 · on screen:** S2 or S3 wins, and every committed entry survives.
> So S2 or S3 becomes leader, and nothing committed is lost. If it takes a while, shift-click S2.

> **CLICK 5 · on screen:** Why it works: committed = on a majority; elected = votes from a majority. They overlap.
> The reason it works: committed means on a majority, winning needs votes from a majority, and those two majorities share a server that refuses.

**Point at:** “refuses vote” lines in the events; no red banner.

**Background:** This is the ‘election restriction’ from §5.4.1 of the paper. S4/S5 keep timing out and bumping the term, which can delay the election; the 4th step nudges S2′s timer (shift-click works too).

## 16. Two leaders? Network partitions

Methods · the artifact, rebuilt live · Demo · about 1:30 min

> **WHEN THE SLIDE OPENS**
> Last failure: the network splits.

> **CLICK 1 · on screen:** The network splits: S1, S2 | S3, S4, S5.
> Red dashed links: S1 and S2 can’t talk to the other three.

> **CLICK 2 · on screen:** S1 still thinks it is leader and accepts a write, but it can’t reach a majority, so it never commits.
> S1 still thinks it’s leader and accepts a write, but only S2 can store it, so it stays faded and uncommitted. The client never gets OK.

> **CLICK 3 · on screen:** The majority side elects a leader in a higher term and commits new writes.
> The other side elects its own leader in a higher term. Two crowns at once. That’s fine: only one can commit.

> **CLICK 4 · on screen:** The network heals. S1 sees the higher term, steps down, and its uncommitted entry is replaced.
> Heal it. S1 sees the higher term, steps down, and its uncommitted entry is replaced. Terms tell everyone which leader is current.

**Point at:** Two crowns; S1′s faded entry being replaced.

**Background:** Two crowns at once is fine: only one of them can ever commit. Terms tell everyone which leader is current.

## 17. The subtle one (Figure 8) (optional)

Methods · the artifact, rebuilt live · Bonus problem · about 1:30 min

**WHEN THE SLIDE OPENS**

Optional, a famous corner case from the paper, Figure 8.

**CLICK 1 · on screen:** Earlier: S1 wrote entry 2 in term 2; S5 wrote a different entry 2 in term 3 and went down. S1 is leader again in term 4.

The history is already set up: two short-lived leaders wrote different entries at index 2.

**CLICK 2 · on screen:** S1 copies its old term-2 entry to a majority and counts it as committed.

S1 copies its old entry to a majority and calls it committed.

**CLICK 3 · on screen:** S1 crashes. S5 returns: its last term (3) beats everyone's (2), so it is allowed to win…

S1 crashes. S5's last entry has term 3, newer than everyone's term 2, so it is allowed to win.

**CLICK 4 · on screen:** …and overwrites the "committed" entry 2 everywhere.

And it overwrites the entry we called committed. Red banner.

**Background:** Figure 8 in the paper. Counting replicas is not enough for entries from an older term.

## 18. Only count replicas for the current term (optional)

Methods · the artifact, rebuilt live · Bonus fix · about 1:00 min

**WHEN THE SLIDE OPENS**

Optional. The fix is one more rule.

**CLICK 1 · on screen:** A leader only commits by counting replicas for entries from its own term.

Only count copies for entries from your own term.

**CLICK 2 · on screen:** S1 copies entry 2 to a majority, but it stays uncommitted.

S1 copies entry 2 to a majority, but does not call it committed yet.

**CLICK 3 · on screen:** S1 commits a new term-4 entry; that commits entry 2 with it.

S1 commits a new entry from its own term; everything before it becomes committed too.

> **CLICK 4 · on screen:** Now S5's last term (3) is behind the majority's (4): it can't win.
>
> Now S5 is behind the majority and can't win.

**Background:** Older entries get committed indirectly, via the log matching property.

## 19. How the authors evaluated Raft

Methods · evaluation · Methods · about 1:00 min

> **WHEN THE SLIDE OPENS**
>
> That was the algorithm. Now: how did the authors show it is actually better?

> **CLICK 1 · on screen:** Understandability: a user study. Each student watched a lecture on both algorithms and took a quiz on each; half saw Paxos first.
>
> 43 students at Stanford and Berkeley, video lectures and a quiz on each algorithm. Half did Paxos first, so order doesn't decide the result.

> **CLICK 2 · on screen:** Correctness: a formal specification and proofs.
>
> A formal TLA+ specification and proofs.

> **CLICK 3 · on screen:** Performance: their own implementation, repeatedly crashing the leader of a 5-server cluster and timing the election.
>
> Their own implementation: crash the leader again and again and time the election.

**Background:** Section 9 of the paper. Note the study was designed to favor Paxos where it could: 15 of 43 had prior Paxos experience, and the Paxos video was 14% longer.

## 20. Students understood Raft better

Results · Results · about 1:30 min

> **WHEN THE SLIDE OPENS**
>
> The user study results. Point at the chart: 25.7 versus 20.8 out of 60.

> **CLICK 1 · on screen:** Raft scored 4.9 points higher on average; 33 of 43 students did better on Raft.
>
> 33 of 43 students did better on Raft.

> **CLICK 2 · on screen:** t-test: with 95% confidence, Raft's true mean is at least 2.5 points higher.
>
> The difference is statistically significant: at least 2.5 points with 95% confidence.

> **CLICK 3 · on screen:** Regression (prior Paxos experience, order): +12.5 points for Raft, but 6.3 lower on Raft if Paxos came first.
>
> The regression controls for prior Paxos experience. Strange detail: learning Paxos first made Raft scores lower, and the authors don’t know why.

> **CLICK 4 · on screen:** Survey: 33 of 41 said Raft would be easier to implement and to explain.
>
> The authors themselves say this survey is weaker evidence than the quiz.

**Background:** The authors themselves say the survey answers are less reliable than quiz scores, and could be biased by knowing the hypothesis.

## 21. Correctness: specified, (mostly) proven

Results · Results · about 1:00 min

> **CLICK 1 · on screen:** Raft is specified in TLA+; one key property is machine-checked, the main safety property has a careful informal proof.
>
> TLA+ is a language for writing exact specifications. 400 lines for Raft, and a 3500-word proof of the main safety property.

> **CLICK 2 · on screen:** Not everything is machine-checked: the Log Completeness proof relies on invariants that were not checked mechanically.
>
> Be honest about the limit: the machine-checked proof rests on invariants that were not machine-checked.

**Background:** Section 9.2.

## 22. Leader election is fast, with randomness

Results · Results · about 1:00 min

> **CLICK 1 · on screen:** Normal operation: one round trip from the leader to a majority per entry, the same as Paxos.
>
> In normal operation each entry takes one round trip to a majority, same as Paxos.

> **CLICK 2 · on screen:** Without randomized timeouts, elections took over 10 seconds: the split vote you saw earlier.
>
> Without randomness, elections took over ten seconds. That’s exactly the endless split vote we showed.

> **CLICK 3 · on screen:** A little randomness fixes it; lower timeouts mean faster failover, until heartbeats can’t keep up.
>
> 5 ms of randomness gives a 287 ms median. Shorter timeouts mean faster failover, until heartbeats can’t arrive in time.

**Background:** Section 9.3, Figure 16. The leader was crashed at a random point within its heartbeat interval.

## 23. What the authors conclude

Discussion · Discussion · about 1:00 min

**WHEN THE SLIDE OPENS**

This slide is only the authors' claims, not ours.

**CLICK 1 · on screen:** Raft is as efficient as Paxos, and easier to understand: the user study supports this.

Same performance as Paxos, easier to understand.

**CLICK 2 · on screen:** Designing for understandability is a goal you can actually test.

Understandability can be designed for and measured.

**CLICK 3 · on screen:** Performance optimizations from Paxos variants could apply to Raft; they leave this to future work.

Further optimizations are future work.

**Background:** Keep this slide strictly to the authors' claims. Our own opinions are on the next slide.

## 24. What we think

Discussion · Our reflections · about 2:00 min

**WHEN THE SLIDE OPENS**

Now our own opinions.

**CLICK 1 · on screen:** The study is careful (order balanced, statistics, materials published), but a quiz after a one-hour lecture measures how teachable Raft is, not how easy it is to build correctly.

The study is well designed, but a quiz after a one-hour video measures how easy it is to teach, not how easy it is to build correctly.

**CLICK 2 · on screen:** Survey answers may be biased: participants could have known the hypothesis.

The survey could be biased: participants could have known what the authors hoped for.

> **CLICK 3 · on screen:** Only part of the proof is machine-checked, and the performance section compares election settings, not Raft against Paxos.
>
> Some proof invariants are unchecked, and there is no head-to-head performance comparison with Paxos.

> **CLICK 4 · on screen:** The strongest evidence came later: Raft is used in widely deployed systems.
>
> The best evidence came after the paper: etcd, the store behind Kubernetes, Consul, CockroachDB and TiKV all use Raft.

> **CLICK 5 · on screen:** Building this demo: the happy path is simple; the subtle part is the safety corner cases. Understandable does not mean trivial.
>
> From building this demo: the normal case is easy. The safety corner cases are where all the subtlety is.

**Background:** Make clear these are our opinions, not the paper’s.

## 25. That’s Raft

Summary · Recap · about 1:00 min

> **WHEN THE SLIDE OPENS**
>
> These are all the rules we added. Each one came from a failure you saw. We skipped snapshots, membership changes, and client details. Thank you.

**Background:** Point at how each rule came from a failure we saw. Mention the parts we skipped.

## 26. Sandbox

Questions · Try it · about 2:00 min

> **WHEN THE SLIDE OPENS**
>
> Questions? We can reproduce any scenario live here: click servers to crash them, split the network, send writes.

**Background:** Take questions; reproduce scenarios live.